`ComplexGitSync` 0002.25– Getting Started

Nicolas Flipo

September 2, 2026

Contents

1 Getting Started

This chapter guides a new user through the Multi-repo Sync CLI. The documented entry point is `cgitsync`.

The primary lifecycle is:

1. `initialise(.cgs)` → clone all repositories → `READY`
2. `initialise(.gts)` → load or restore a saved snapshot → `READY`
3. `pull(.cgs|.gts)` → resynchronise an existing tree
4. `checkout` / `add` / `commit` / `push` / `tag` → tree-wide Git operations
5. `freeze` → write the next `.gts` snapshot and update the project-local `.lgr`

2 Prerequisites

- Python 3.11 or later.
- Git 2.30 or later available on `PATH`.
- Working Git authentication for every remote used by the workspace.

3 Installation

Install `ComplexGitSync` from source with the Pixi-managed environment:

```
$ git clone https://github.com/flipoyo/ComplexGitSync.git
$ cd ComplexGitSync
$ pixi install
$ pixi run cgitsync --help
```

4 Create or Check a Project Spec

A `.cgs` file is the local authoring spec for the repository tree. It normally contains only a project name and repository identifiers:

```
project = "CGSil1"
repos = [
  "gitlab:CGS_test/CGSil1",
  "gitlab:CGS_test/CGSil2",
  "github:flipoyo/CGSih1",
]
```

Each identifier uses `provider:owner/repository`. ComplexGitSync runs `PARSE -> NORMALIZE -> VALIDATE` and supplies deterministic defaults: `main` branches, `ssh`, `nested_config = auto`, and the repository name as its path. A unique repository matching the project name is mounted at `.`. Advanced tables are reserved for overrides.

Canonical providers are `github`, `gitlab`, `codeberg`, and `custom`. Their known hosts are `github.com`, `gitlab.com`, and `codeberg.org`; SSH and HTTPS remotes are generated automatically. Custom hosting requires an explicit `gitprovider_url` and never guesses a host. For example, `codeberg:GX4G/GX4G` identifies owner `GX4G` and repository `GX4G` on Codeberg.

Copy `examples/template.cgs` when starting a specification. It combines plain repository identifiers with one inline override. Developers can compare it with `examples/normalized_template.cgs`, which shows the complete canonical document after normalization and is not intended as the usual hand-authored form.

The common inline options are `default_branch`, `fallback_branch`, `access_protocol`, `relative_path`, and `nested_config`. A relative path is resolved from the parent repository. Nested config can be `auto` to load the sole root-level `*.cgs`, `disabled` to stop discovery, or a relative `.cgs` path to select an exact file inside the repository.

```
$ pixi run cgitsync validate examples/complexgitsync.cgs
$ pixi run cgitsync view-tree examples/complexgitsync.cgs
```

A valid declared tree prints a lifecycle line such as:

```
DECLARED ready=false complete=true
```

5 Initialise the Workspace

Use `initialise` as the canonical first command. From a `.cgs` file it clones the full repository tree, validates it, writes the runtime snapshot under `<workspace>/.cgitsync/state/`, and leaves the tree `READY`.

```
$ pixi run cgitsync initialise examples/complexgitsync.cgs
```

The same canonical project definition can be authored directly. The `-repo` option is repeatable:

```
$ pixi run cgitsync initialise --project CGSil1 \
    --repo gitlab:CGS_test/CGSil1 \
    --repo codeberg:GX4G/GX4G
```

Use either a source file or `-project` plus `-repo`, never both. To serialize the CLI definition without initializing a workspace, use `create-cgs -project ... -repo ... -output FILE`.

When `-output-path` is omitted, `cgitsync` uses the workspace-level default `CGSPATH=../..`, then derives `CGSHOME=CGSPATH/<project-name>`. The explicit equivalent is:

```
$ pixi run cgitsync initialise examples/complexgitsync.cgs --output-path
    "$CGSPATH"
```

From a previously generated snapshot (resolved automatically via the project's `.lgr` register; pass the path explicitly only to override that discovery):

```
$ pixi run cgitsync initialise
```

Successful `.cgs` initialisation prints a readable operation sequence `GT-LOAD->GT-DISCOVER->GT-VALIDATE->` the explicit workflow `load->expand->validate->clone->gitignore`, the per-repository `git clone` action, the resolved root path, a report of any `.gitignore` changes, and a compact tree outline. Structured log records also put the operation code first, for example `GT-DISCOVER`, `GT-VALIDATE`, `FS-PURGE`, `GT-CLONE`, or `GT-GITIGNORE`. If a stale partial checkout blocks the clone step, `initialise` fails explicitly and prints `Try clean-init method`. Use `clean-init` to insert a purge phase between validation and cloning:

```
$ pixi run cgitsync clean-init examples/complexgitsync.cgs
```

The explicit operation sequence is `GT-LOAD->GT-DISCOVER->GT-VALIDATE->FS-PURGE->GT-CLONE->GT-GIT` the matching workflow is `load->expand->validate->purge->clone->gitignore`. The purge phase can also be run on its own:

```
$ pixi run cgitsync purge examples/complexgitsync.cgs
```

`purge` removes generated clone state from `CGSHOME`: immediate child repository directories declared directly under the project root, and project `*.lgr` files. A persisted Git identity override at `.cgitsync/master.toml` (see the `initialise` entry in the Command Reference chapter for `-git-user-name`/`-git-user-email`) is workspace configuration, not clone state, and is left untouched.

6 Inspect the Runtime State

```
$ pixi run cgitsync view-tree
$ pixi run cgitsync view-tree --depth 2 --collapse parent_2
$ pixi run cgitsync status
```

If a command accepts an optional snapshot and none is provided, `cgitsync` discovers the latest `.gts` automatically via the project's `.lgr` register under `CGSHOME/.cgitsync/`. Each generated `.gts` snapshot actually lives under its own content-addressed `CGSHOME/.cgitsync/state(<hash>)_<n>/` directory; pass an explicit path (e.g. `-gts FILE`, or a positional source, depending on the command) only to override that discovery. `CGSHOME` can be set explicitly; for `initialise(.cgs)` the default is `CGSPATH=../..`, and later commands can also discover the workspace by walking upward from the current directory.

7 Run the Git Cycle

Once the tree is `READY`, the Multi-repo Sync commands operate on every repository in deterministic order:

```
$ pixi run cgitsync pull
$ pixi run cgitsync checkout feature/my-branch
$ pixi run cgitsync add
$ pixi run cgitsync commit "feat: update project CGS#1"
$ pixi run cgitsync push
$ pixi run cgitsync freeze-release release-2026.05 "release 2026.05"
$ pixi run cgitsync freeze release-2026.05
$ pixi run cgitsync launch-release release-2026.05
```

`pull` walks the tree parent-first, including the project root: `root -> parent -> leaf`. Mutation commands such as `add`, `commit`, `push`, and `freeze` walk the opposite direction: `leaf -> parent -> root`. When local files block the safe pull, the CLI suggests `cgitsync pull-force`; that recovery command is destructive and discards local uncommitted/untracked work before force-updating the same tree.

Pass `-gts <snapshot.gts>` when you want to pin a command to an explicit snapshot, and pass `-dry-run` to `add`, `commit`, `push`, or `freeze` to preview the plan without mutating repositories.

8 Log Files

Every CLI command writes a timestamped log file. On Linux the default location is `/.local/state/ComplexGi` if `XDG_STATE_HOME` is set, logs are written under that state directory instead. The CLI prints the resolved path as `log_file=<path>`.

9 Next Steps

See Chapter **??** for the command reference, document formats, preflight checks, and troubleshooting notes.